OpenTelemetry Fundamentals

> **Series:** OTEL | **Notebook:** 1 of 8 | **Created:** January 2026

Introduction to OpenTelemetry and Dynatrace

OpenTelemetry (OTel) is the industry-standard framework for collecting telemetry data. Dynatrace fully supports OpenTelemetry through native OTLP ingestion, allowing you to leverage OTel instrumentation while benefiting from Dynatrace's AI-powered analytics.

Table of Contents

1. What is OpenTelemetry?
2. OTel vs. Proprietary Agents
3. Core Components
4. Signal Types: Traces, Metrics, Logs
5. Semantic Conventions
6. Dynatrace OTel Integration
7. When to Use OpenTelemetry
8. Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with OTLP ingestion enabled
Permissions	`openpipeline.events`, `metrics.ingest`, `logs.ingest`
Knowledge	Basic observability concepts

1. What is OpenTelemetry?

OpenTelemetry is a vendor-neutral, open-source observability framework that provides:

Component	Purpose
APIs	Standardized interfaces for instrumentation
SDKs	Language-specific implementations
Collector	Agent for receiving, processing, and exporting telemetry
Protocol (OTLP)	Standard wire protocol for telemetry data

CNCF Project

OpenTelemetry is a Cloud Native Computing Foundation (CNCF) project, formed from the merger of OpenTracing and OpenCensus. It's the second-most active CNCF project after Kubernetes.

Key Benefits

Benefit	Description
Vendor Neutral	No lock-in to specific backends
Standardized	Consistent across languages and platforms
Comprehensive	Traces, metrics, and logs in one framework
Community-Driven	Active development, broad adoption

2. OTel vs. Proprietary Agents

Comparison

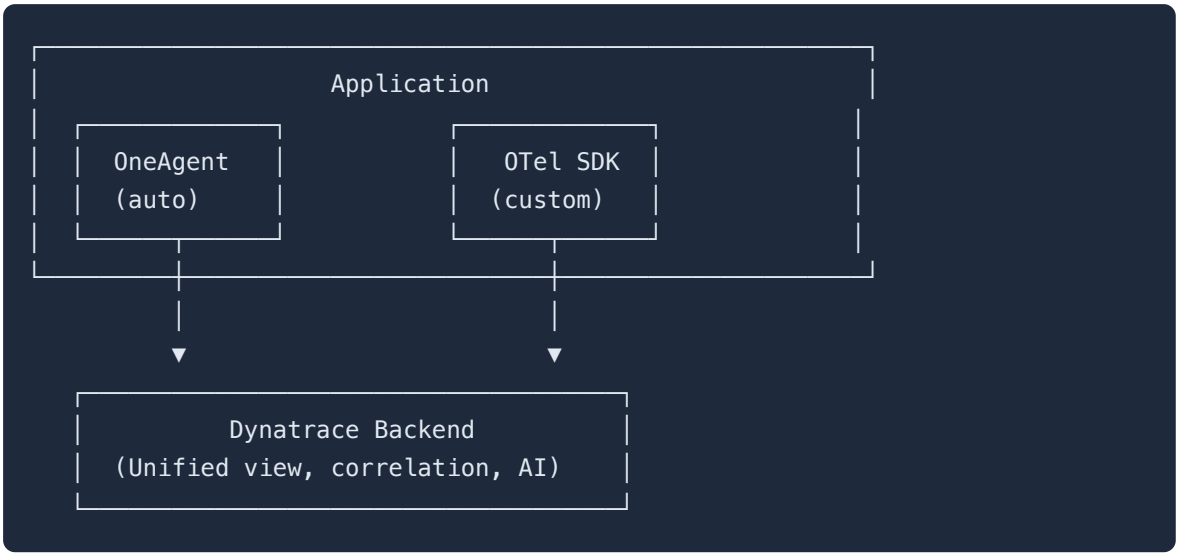
Aspect	OpenTelemetry	Dynatrace OneAgent
Instrumentation	Manual or auto-instrumentation	Automatic
Deployment	SDK + Collector	Single agent
Coverage	Code-level	Full-stack
Flexibility	High (any backend)	Dynatrace-specific
Maintenance	Higher	Lower
Features	Standard signals	AI, RCA, Davis

When to Choose OTel

Scenario	Recommendation
Multi-vendor observability	OTel
Serverless/Lambda	OTel
Custom instrumentation needs	OTel
Existing OTel investment	OTel
Full-stack visibility	OneAgent + OTel
Minimal effort	OneAgent

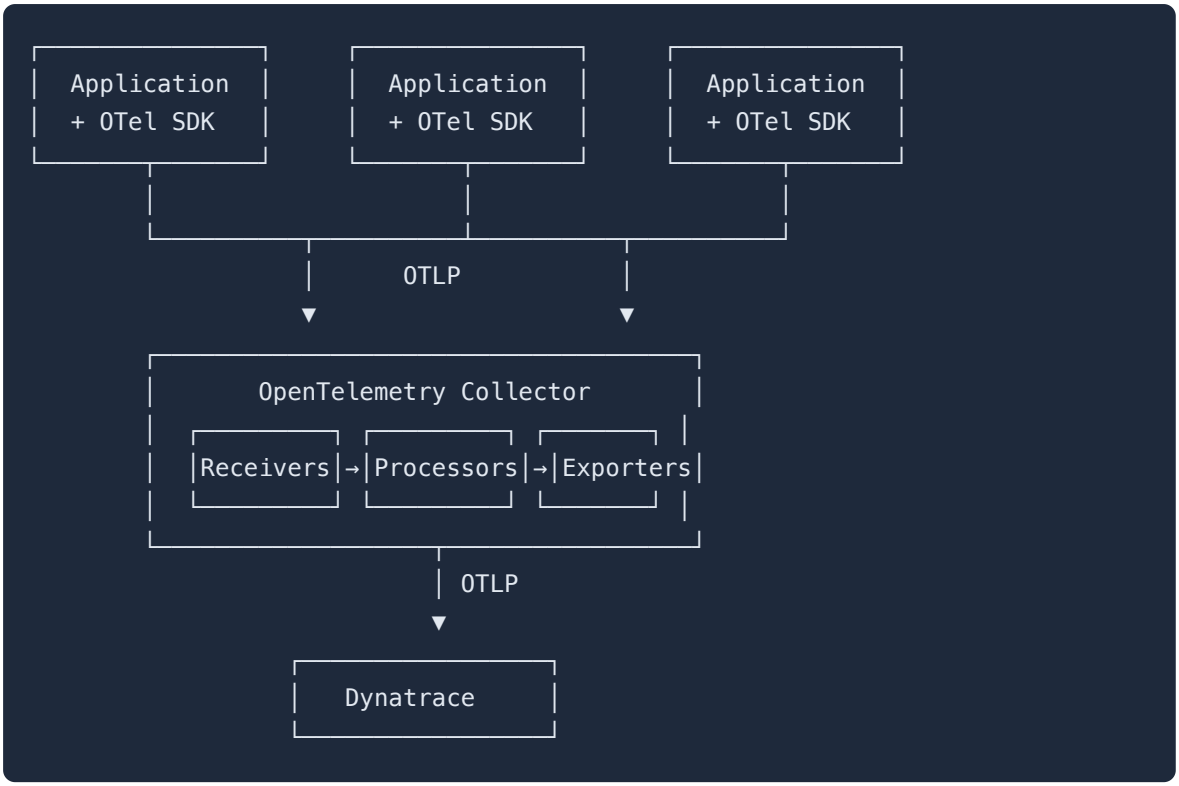
Hybrid Approach

Dynatrace supports using both:



3. Core Components

OpenTelemetry Architecture



Component Descriptions

Component	Role
API	Defines how to instrument (language-specific interfaces)
SDK	Implements the API, handles sampling, batching
Exporter	Sends data to backends (OTLP, Jaeger, etc.)
Collector	Standalone service for processing telemetry
Instrumentation Libraries	Pre-built instrumentation for frameworks

4. Signal Types: Traces, Metrics, Logs

Traces

Distributed traces track requests across services.

Concept	Description
Trace	End-to-end transaction path
Span	Single operation within a trace
SpanContext	Propagated context (trace ID, span ID)
Attributes	Key-value metadata on spans
Events	Timestamped annotations on spans

Metrics

Quantitative measurements over time.

Instrument Type	Use Case	Example
Counter	Monotonically increasing value	Request count
UpDownCounter	Value that goes up and down	Active connections
Histogram	Distribution of values	Response time
Gauge	Current value	Temperature, queue size

Logs

Structured event records correlated with traces.

Field	Description
Timestamp	When the event occurred
Severity	Log level (DEBUG, INFO, ERROR)
Body	Log message content
Attributes	Structured metadata
TraceContext	Link to active span

5. Semantic Conventions

Semantic conventions define standard attribute names for consistent telemetry.

Resource Attributes

Attribute	Example	Description
`service.name`	`checkout-api`	Logical service name
`service.version`	`1.2.3`	Service version
`service.namespace`	`production`	Service namespace
`deployment.environment`	`prod`	Deployment environment

HTTP Span Attributes

Attribute	Example	Description
`http.method`	`GET`	HTTP method
`http.url`	`https://api.example.com/users`	Full URL
`http.status_code`	`200`	Response status code
`http.route`	`/users/{id}`	Route template

Database Span Attributes

Attribute	Example	Description
`db.system`	`postgresql`	Database type
`db.name`	`users_db`	Database name
`db.statement`	`SELECT * FROM users`	Query
`db.operation`	`SELECT`	Operation type

Kubernetes Attributes

Attribute	Example	Description
`k8s.namespace.name`	`checkout`	Namespace
`k8s.pod.name`	`checkout-api-abc123`	Pod name
`k8s.deployment.name`	`checkout-api`	Deployment

6. Dynatrace OTel Integration

OTLP Endpoints

Dynatrace accepts OTLP data via:

Protocol	Endpoint
gRPC	`https://{your-env}.live.dynatrace.com/api/v2/otlp`
HTTP	`https://{your-env}.live.dynatrace.com/api/v2/otlp`

Authentication

Use Dynatrace API token with required scopes:

Signal	Required Scope
Traces	`openTelemetryTrace.ingest`
Metrics	`metrics.ingest`
Logs	`logs.ingest`

Configuration Example

OTel Collector Exporter:

```
exporters:
  otlphttp:
    endpoint: https://{your-env}.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: Api-Token dt0c01.xxx...
```

SDK Direct Export (Python):

```
from opentelemetry.exporter.otlp.proto.http.trace_exporter import
OTLPSpanExporter

exporter = OTLPSpanExporter(
    endpoint="https://{your-env}.live.dynatrace.com/api/v2/otlp/v1/traces",
    headers={"Authorization": "Api-Token dt0c01.xxx..."}
)
```

```
// View OpenTelemetry traces in Dynatrace
fetch spans
| filter isNotNull(otel.library.name)
| fields timestamp, trace.id, span.name, otel.library.name, duration
| sort timestamp desc
| limit 20
```

```
// OTel instrumentation libraries in use
fetch spans
| filter isNotNull(otel.library.name)
| summarize count(), by:{otel.library.name, otel.library.version}
| sort count() desc
| limit 20
```

7. When to Use OpenTelemetry

OTel is Ideal For

Scenario	Why OTel
Serverless	AWS Lambda, Azure Functions (no agent)
Multi-cloud	Consistent instrumentation across clouds
Custom metrics	Business-specific measurements
Vendor diversity	Send to multiple backends
Edge/IoT	Lightweight collection
Polyglot	Many languages, one standard

OneAgent is Better For

Scenario	Why OneAgent
Full-stack	Infrastructure + code in one
Zero-effort	Auto-instrumentation
Davis AI	Full AI capabilities
PurePath	Complete transaction tracing
Real User Monitoring	RUM integration

Recommended: Hybrid

For most enterprises:
- **OneAgent** for infrastructure and supported technologies
- **OTel** for unsupported languages, serverless, custom metrics

Next Steps

Now that you understand OTel fundamentals, proceed to:

Next Notebook	Topic
OTEL-02: Collector Architecture	Deep-dive into the Collector
OTEL-03: Collector Deployment	Deployment patterns
OTEL-04: Trace Instrumentation	Instrumenting your code

Summary

In this notebook, you learned:

- What OpenTelemetry is and its benefits
- How OTel compares to proprietary agents
- Core OTel components: API, SDK, Collector
- Signal types: traces, metrics, logs
- Semantic conventions for consistent telemetry
- Dynatrace OTLP integration configuration
- When to use OTel vs. OneAgent

References

- OpenTelemetry Official Docs
- Dynatrace OpenTelemetry Integration
- OTLP Specification
- Semantic Conventions
